A PLAIN-LANGUAGE GUIDE

AI Agents

What they are, how they work, and how to use them well — explained without the jargon.

A friendly introduction · No technical background required

Made by Kaio to the CS Team ❤

1 What Is an AI Agent?

A regular chatbot is like a vending machine: one request in, one answer out, done. An **AI agent** goes further — it takes a **goal**, breaks it into steps, decides what to do, uses tools to get information or take action, checks the result, and keeps going until the job is finished.

The cleanest dividing line: **if a system only talks, it's a chatbot. If it can decide what to do next and take action, it's an agent.** A chatbot focuses on the conversation; an agent focuses on the *outcome*.

> *Ask a chatbot "what's the status of invoice #4521?" and it answers. Give an agent "resolve this billing dispute" and it reads the ticket, pulls the order, checks policy, drafts a reply, and flags anything odd — a sequence of steps, not a single reply.*

2 How an Agent Works: The Loop

Under the hood, agents run a simple repeating cycle — the **agent loop**. There are four intuitive moves that repeat until the task is done:

- **Perceive** — take in the request and any new information (a ticket, a search result, an error).
- **Think / Plan** — decide the next step: "I need the order date, so I'll look up the account."
- **Act** — do something, usually by *using a tool* (search the knowledge base, query the order system).
- **Observe** — read the result and feed it into the next round of thinking.

The power isn't in any single step — it's in the **iteration**. Because each result becomes the next input, the agent can notice when something went wrong and self-correct. That's what turns AI from something that *answers* into something that *works*.

3 What's Inside an Agent

Think of an agent as a capable new employee. It has four parts:

- **The LLM "brain"** — the reasoning engine that interprets language, makes plans, and decides what to do. On its own, it's "a brilliant mind in a jar" — it can think but not act.
- **Tools (the "hands")** — what lets it act: search the web, look up an order, send an email, query a database. The agent decides *when* a tool is needed.
- **Memory** — short-term memory tracks the current task; long-term memory stores useful facts and past interactions, so you don't re-explain everything.
- **Goals / instructions** — the "job description." The clearer the goal and the guardrails, the better it performs.

WORTH KNOWING

A standard called **MCP (Model Context Protocol)** acts like a universal adapter, letting agents plug into many different tools and systems — increasingly the common "plug" across platforms.

4 How Agents Remember

Memory is what separates an agent that feels capable from one that forgets you every time. Agents juggle a few different kinds, much like people do:

- **Working (short-term) memory** — what's in front of it right now: the current task and conversation. Like the notes on your desk. It's limited by the context window, and when it fills up, the oldest details fall off.
- **Semantic memory** — stable facts it has learned and kept: "this customer prefers email," "our refund window is 30 days." Its organized encyclopedia about a user or topic.
- **Episodic memory** — specific past events: "last week this customer had a billing issue we solved by X." It lets the agent recall what worked before and build continuity across sessions.
- **Procedural memory** — how to do recurring things: the steps of a password reset, the flow of a refund. A support agent on its hundredth reset doesn't re-think it — it runs a learned procedure.

> *Working memory is the sticky note on your monitor; the long-term kinds are the filing cabinet, the diary, and the muscle memory. Good agents know which drawer to open.*

Without long-term memory, an agent is "stateless" — bright but forgetful, re-introducing itself every time. With it, the agent gets more personal and more efficient the more you use it.

5 Levels of Autonomy

Borrowing from self-driving cars: autonomy is a **dial, not a switch**. Agents range from "just suggests" to "acts on its own."

> **Assistant** — you do the work; the AI answers questions or suggests a line.

> **Copilot** — the AI drafts and recommends, but a human approves and executes every step.

> **Supervised agent** — it runs low-risk steps on its own and asks for approval at key checkpoints.

> **Autonomous agent** — it plans and acts toward a goal with minimal human involvement.

A related idea: **predefined workflows** follow fixed steps you wrote in advance, while **autonomous agents** decide the steps themselves. Most useful real-world agents sit in the *middle* of the dial — supervised, not fully autonomous.

6 Agents You'll Meet

- **Coding agents** — the most widely used category; they write, debug, and test code (Claude Code, Cursor, GitHub Copilot).
- **Research agents** — explore a topic, read many sources, and synthesize a report.
- **Customer support agents** — triage tickets, fetch order/account data, draft replies, resolve common issues.
- **Personal assistants** — manage email, calendars, and scheduling.
- **Computer-use agents** — actually "see" a screen and click, type, and navigate like a person, operating software with no tidy integration.

7 Using Agents Well

The most useful frame: **treat an agent like a capable new intern.** You wouldn't expect an intern to deliver on day one without explaining the task, the tools, and the format. Same here:

Write a clear goal

State the why, the what, and the scope. Vague goals produce vague results.

Give context & tools

Point it to the knowledge base, the order system, the policy doc. "Check here" beats "figure it out."

Scope it tightly

Single-purpose agents that do one thing well beat "do-everything" agents.

Break it down

Don't dump a five-part mega-request at once; stage it.

Iterate

Test, see where it goes wrong, refine the instructions. It's trial and error.

Start simple

Only add complexity when the task genuinely needs it.

8 Staying in Control

- **Human-in-the-loop checkpoints** — insert review points where a person approves, corrects, or rejects before an action takes effect.
- **Match oversight to risk** — let the agent run freely on low-risk steps; require approval for high-consequence ones (anything touching money, data, or customers).
- **Set guardrails** — give the agent only the tools and data it truly needs ("least privilege"), and define escalation paths.
- **Know when to step in** — if a decision is irreversible or sensitive, a human approves before the agent acts.

One nuance: a rubber-stamp "Approve" button doesn't make someone a real error-catcher. Reviewers need to see the *actual* action and its consequences, and focus attention on the genuinely high-stakes cases.

9 When to Use One (and When Not To)

KEEP IT SIMPLE WHEN…

the task is short, predictable, low-risk, and linear — FAQs, password resets, pricing questions, simple lookups. A plain chatbot or fixed workflow is cheaper, faster, and more predictable.

REACH FOR AN AGENT WHEN…

the task is genuinely multi-step, needs several tools or systems, involves judgment about what to do next, or is a repetitive workflow worth automating.

Rule of thumb: ***workflows for the predictable, agents for the unpredictable.***

10 What Goes Wrong

- **Hallucinations** — agents can be confidently wrong. And when an agent *acts* on a hallucination, it doesn't just say something wrong, it *does* something wrong.
- **Compounding errors** — across many steps, a small early mistake snowballs as each wrong step feeds the next.
- **Cost & latency** — agents "think" across many steps, so they're slower and pricier than a single reply.
- **Over-trust** — because they sound fluent, people accept outputs without checking.
- **Security & privacy** — watch for **prompt injection**, where hidden instructions in a webpage, document, or even a customer message trick an agent into leaking data or taking unauthorized actions.

THE COMPOUNDING MATH, MADE CONCRETE

Because each step must succeed for the whole task to succeed, you *multiply* the success rates. At a strong 90% per step, ten steps in a row succeed only about **35%** of the time (0.90^{10}); at 95%, about **59%**. This is why short, checked chains beat long, hopeful ones — and why cost can balloon too: a heavy multi-step agent can use far more processing than a single reply, sometimes by a large multiple.

A REAL CAUTIONARY TALE

In 2024, a tribunal ordered **Air Canada** to pay a customer after its support chatbot invented a refund policy that didn't exist. The airline argued the chatbot was "responsible for its own actions" — and lost. The lesson: a company is fully accountable for what its AI tells customers.

11 In Real Life: A Support Ticket

A customer writes: *"I was charged twice and my service is down."* An agentic support system might:

Classify → **Pull order & account** → **Draft reply** → **Human reviews & sends**

It reads and classifies the ticket, pulls the customer's billing and service data, drafts a clear reply grounded in approved help docs, and hands it to a rep to review and send. For an action like issuing a refund, a human approval gate sits in front of it.

The safest starting pattern

The agent **drafts**, a human **reviews and sends**. Lowest risk, high value — and a great way to measure how often reps need to rewrite the draft.

12 The Landscape & Where It's Heading

There's a crowded but converging ecosystem in 2026: big labs (OpenAI, Anthropic, Google), productivity giants (Microsoft Copilot, Salesforce Agentforce), developer frameworks (LangGraph, CrewAI, AutoGen), and a shared connection standard (MCP) that reduces lock-in.

The clear direction is toward **more capable, more autonomous agents** and **teams of specialized agents** coordinated by an orchestrator. For support teams, the likely shape is *"bounded autonomy"*: agents handle execution while humans keep judgment, accountability, and oversight. The emerging skill for everyone becomes **supervising** agents rather than doing every step by hand.

STAY SKEPTICAL

Many flashy "resolution rate" numbers come from vendors and aren't independently audited — one famous company that replaced support staff with AI later rehired humans over quality. "Agent" is also a loose label slapped on everything. Judge a tool by what it actually *does*, and pilot on your own data.

The Golden Rule: Treat an agent like a capable intern — give it a clear goal, the right tools, and firm limits; let it handle the legwork; and always keep a human on the decisions that matter. Start simple, and add autonomy only when it earns it.